
Big Data

Management and Analytics

TU Clausthal, Institut für Informatik

Summer Semester 2026

Due Date: 24 June 2026, 16:00

Instructions - Please Read Carefully

In this assignment, we study graph matrices, spectral graph analysis, and frequent pair mining. In Task 1, you work with a directed web graph, construct adjacency, degree, and Laplacian matrices, compute eigenvalues and eigenvectors, and reason about spider traps and dead-ends. In Task 2, you study the PCY algorithm and explain how hashing can reduce the number of candidate pairs counted in the second pass compared with Apriori.

Show all essential steps and calculations clearly. When Python is requested, include the relevant code and results. If information is missing, state your assumptions explicitly.

Assignment Rules

- The assignment has a total of **10 marks**
- Work in your assigned teams of **up to 2 members**
- Submit your solutions **via Moodle**
- Solutions submitted after the deadline will automatically receive **0 marks**
- Copying solutions from or sharing with other students will lead to **exclusion from the course**
- The use of language software is **not permitted**; each group member must understand every part of the solution
- Document your solutions with **screenshots** and attach a **TXT file** containing only code and queries, not written solutions
- Solutions generated by large language models will automatically lead to **exclusion from the course**
- If anything is unclear, state your assumptions explicitly to make your solution understandable

Task 1 5 Marks

Suppose we are given the following **directed graph**, consisting of **9 nodes** and **21 edges**. The nodes can be interpreted as websites, and the edges are the hyperlinks connecting them.

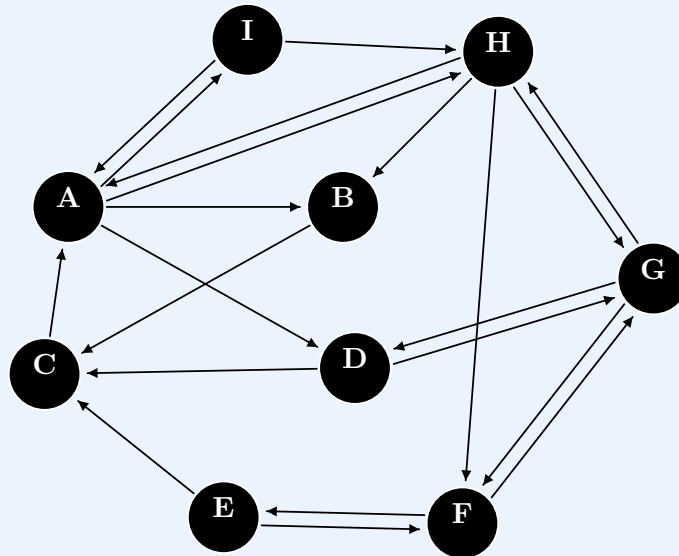


Fig. 1: G

- Write down the adjacency matrix A and the degree matrix D . Afterwards, write a short Python program to generate these matrices in your Python script. Consider the edges in the graph as **undirected** for this task.
- Calculate the Laplacian matrix for the subgraph consisting of the nodes D , E , F , and G , and the relationships between them. Afterwards, calculate the eigenvalues and eigenvectors **without Python**. Consider the edges in the graph as **undirected** for this task.
- Calculate the Laplacian matrix for the whole graph. Use the matrices that you implemented in part (a) with Python. Calculate the eigenvalues and eigenvectors **with Python**. Consider the edges in the graph as **undirected** for this task.
- Calculate the eigenvalues and eigenvectors for your matrix L **with Python**. Interpret your results.
- Remove the minimum number of edges to create a **spider trap** in the graph, but no dead-end. Explain your method and present your results.
- Can you produce a spider trap or dead-end by adding edges in a connected graph? If yes, show an example of such a graph before and after you added edges to create a spider trap or dead-end.

Task 2 5 Marks

This task focuses on the **PCY algorithm** for frequent pair mining.

Items: 1 = Toyota, 2 = Honda, 3 = Ford, 4 = BMW, 5 = Tesla. Use these IDs in the hash function.

Dataset:

{Toyota, Honda, Ford}, {Toyota, Honda, BMW}, {Honda, Ford, BMW},
{Toyota, Honda, Ford}, {Ford, BMW, Tesla}, {Toyota, Ford, Tesla}.

The support threshold is $s = 3$. Use the hash function $h(i, j) = (i \times j) \bmod 7$ with 7 buckets.

- (a) In your own words, explain the **PCY algorithm**. Describe how PCY improves the Apriori approach by using a hash table to reduce the number of candidate pairs that need to be counted in the second pass.
- (b) Count the support of every individual item. Which items meet the threshold s ? These are your frequent items.
- (c) A pair can only be frequent if both of its items are frequent. Explain in one sentence why this must be true.
- (d) For each basket, list its pairs and compute $h(i, j)$ for each. Tally how many pairs land in each bucket by building a table with two columns: **bucket** and **count**.
- (e) Mark every bucket whose count reaches s as a frequent bucket.
- (f) Explain why a truly frequent pair must always land in a frequent bucket. Then state the contrapositive: what can you conclude about a pair in a non-frequent bucket?
- (g) Two different pairs can hash to the same bucket. Find an example in your tally. Why does this mean a frequent bucket does not guarantee that any pair in it is frequent? This is the false-positive idea.
- (h) State the two conditions a pair must satisfy to be counted on Pass 2.
- (i) List every pair that survives both conditions. Compare this to the full list of pairs Apriori would have counted. How many did PCY eliminate before counting?
- (j) Now actually count the surviving pairs and report which are genuinely frequent. Were any survivors not frequent? Those are the false positives the hash filter let slip through.
- (k) PCY never discards a truly frequent pair. Why is that guarantee important, and which property from part (g) ensures it?
- (l) What happens to the quality of the bucket filter if you use very few buckets? What happens if you use very many buckets? What is the memory cost of using very many buckets?